Herramientas de código abierto para Flaski

Oct 25, 2026 · @Eva

Resumen

Hay unas diez herramientas de código abierto que encajan con Flaski: funcionan en el navegador y sin conexión, y su licencia es compatible con la de la app. Antes de añadir funciones, conviene una que protege lo que ya hay: el modo local guarda todo en `localStorage`, que suele tener un límite de unos 5 MB y, con miles de preguntas, podría dejar de guardar.

Lo que haría ya:

- **idb-keyval:** pasar el modo local a IndexedDB, que tiene mucho más espacio.
- **@axe-core/playwright:** que las pruebas revisen solas la accesibilidad de cada pantalla.
- **Lighthouse CI:** que la prueba de 5.000 preguntas falle si la app se vuelve lenta.
- **ts-fsrs:** la librería oficial de FSRS, ya en la versión 6, en lugar del fsrs.js escrito a mano.

Lo que más cambiaría la app después:

- **Tesseract.js:** hacer una foto al temario y convertirla en apunte.
- **WanaKana, kuroshiro y pinyin-pro:** escribir kana con el teclado normal, y furigana y pinyin automáticos.
- **fsrs-browser:** ajustar los repasos a la memoria de cada persona.
- **GlitchTip:** enterarte de los errores que les salen a otros sin que te los cuenten.

Lo que he descartado: Mermaid, porque ocupa demasiado; PowerSync, porque obliga a cambiar toda la capa de datos y añade un servicio; y Whisper para el dictado, porque la voz del navegador ya cubre casi todo.

Lo que Flaski ya usa

Flaski es JavaScript sin frameworks ni empaquetador, y lleva muy pocas librerías de fuera. Es una virtud que conviene conservar: cada herramienta nueva tiene que funcionar sin conexión, cargarse solo cuando se usa y tener una licencia compatible con la de Flaski (MIT).

Pieza	Para qué	Dónde
Hanzi Writer (MIT)	Escribir kanji y hanzi a mano, trazo a trazo	`js/vendor/hanzi-writer.min.js`

Pieza	Para qué	Dónde
PDF.js	Importar un PDF como apuntes, se carga al usarlo	`js/vendor/pdfjs/`
supabase-js (MIT)	Cuentas y nube	`js/vendor/supabase.js`
fzstd	Descomprimir mazos de Anki modernos	`js/vendor/fzstd.js`
FSRS-5, escrito a mano	Calcular cuándo toca repasar cada pregunta, con los parámetros por defecto	`js/fsrs.js`
Voz del navegador	Leer en voz alta y dictar	`js/tts.js`
Playwright	Pruebas e2e en móvil y escritorio	`tests/`

El resto, como el lector de SQLite para Anki, el zip, los gráficos o los iconos, está escrito dentro de la app.

Repaso y memoria

Flaski calcula los repasos con FSRS-5 escrito a mano y con los parámetros por defecto, iguales para todo el mundo. El equipo de FSRS mantiene dos piezas abiertas que lo mejorarían: su librería oficial, ya en FSRS-6, y un optimizador que ajusta los parámetros a cómo recuerda cada persona.

Herramienta	Qué aporta	Licencia y tamaño	Veredicto
ts-fsrs	La implementación oficial en TypeScript, con FSRS-6. Trae vista previa de las cuatro respuestas, deshacer un repaso (`rollback`), olvidar una pregunta y reprogramar todo con otros parámetros.	MIT. Versión 5.4.2.	**Sí.** Sustituir el fsrs.js escrito a mano por la versión oficial, para seguir sus mejoras sin mantenerlas tú. Con una prueba que compare los intervalos antes y después.

Herramienta	Qué aporta	Licencia y tamaño	Veredicto
fsrs-browser	Ejecuta en el navegador el optimizador oficial (fsrs-rs), que aprende tus parámetros a partir de tu historial de repasos. Su ejemplo entrena con 24.394 repasos en unos segundos.	BSD-3. Versión 6.6.0, unos 390 kB.	**Sí, después de ts-fsrs.** Un botón en Estadísticas, «Ajustar a tu memoria», que solo aparece con suficientes repasos (por ejemplo, desde 1.000). El autor avisa de que no siempre va al día con fsrs-rs.
@open-spaced-repetition/binding	El mismo optimizador para Node.	MIT.	**No** para la app, que no tiene servidor propio. Puede servir en las pruebas, para comprobar que fsrs-browser da lo mismo.

La lista oficial de herramientas de FSRS recoge ts-fsrs como la librería de JavaScript y fsrs-browser como la forma de usar fsrs-rs en el navegador.

Apuntes y contenido

La herramienta que más cambiaría la app aquí es leer el texto de una foto: con ella, una página del temario fotografiada se convierte en un apunte. Las demás cubren huecos más concretos.

Herramienta	Qué aporta	Licencia y tamaño	Veredicto
Tesseract.js	Lee el texto de una foto o de un PDF escaneado, en el navegador y sin servidor. Puede funcionar sin conexión si se alojan sus archivos. No sirve para letra a mano.	Apache-2.0. Unos 15 MB de código y datos, que se descargan la primera vez que se usa.	**Sí.** «Foto del temario» al crear un apunte, y también para los PDF escaneados, que ahora no dan texto. Que se cargue solo al usarlo, como PDF.js.

Herramienta	Qué aporta	Licencia y tamaño	Veredicto
MiniSearch	Buscador de texto completo: por prefijo («const» encuentra «Constitución»), con errores tolerados y más peso a unos campos que a otros. No tiene dependencias.	MIT. Versión 7.2.0.	**Probar** en la prueba de rendimiento con 5.000 preguntas. Si el buscador actual ya responde en menos de 100 ms, no hace falta. Las tildes se resuelven normalizando las palabras.
KaTeX	Dibuja fórmulas matemáticas escritas en LaTeX, rápido y sin servidor.	MIT. Unos 4 MB en npm, pero lo que se carga es mucho menos.	**Cuando alguien lo pida.** Para oposiciones de ciencias, economía o estadística. Carga solo si el apunte o la pregunta tiene `$…$`.
highlight.js	Colorea el código según el lenguaje.	BSD-3. Se puede cargar solo con los lenguajes que hagan falta.	**Opcional.** Las preguntas de código ya existen; colorearlas ayuda a leerlas.
DOMPurify	Limpia HTML para que no se pueda colar código malicioso.	MPL-2.0 o Apache-2.0.	**Solo si** algún día se pinta HTML que viene de otros, por ejemplo de mazos de la comunidad o de Anki. Conviene pedir a Claude Code que confirme que hoy todo pasa por `esc()`.
Mermaid	Diagramas a partir de texto.	MIT, pero ocupa unos 122 MB en npm.	**No.** Es demasiado grande para una app que funciona sin conexión. El mapa de Flaski ya cubre esa necesidad.

Idiomas

Para los mazos de idiomas hay herramientas abiertas que ahorrarían mucho trabajo a quien estudia: furigana y pinyin automáticos, escribir kana con el teclado normal y frases de ejemplo con traducción.

Herramienta	Qué aporta	Licencia y tamaño	Veredicto
WanaKana	Convierte lo que escribes en letras latinas a kana mientras escribes («kanji» → «かんじ»).	MIT. Pequeña.	**Sí.** En las preguntas de escribir la respuesta en japonés, para quien no tiene el teclado japonés instalado. Es lo más barato de esta lista.
kuroshiro con kuromoji	Pone el furigana solo: escribes «漢字» y sale «漢字[かんじ]». También pasa a rōmaji.	MIT (kuroshiro) y Apache-2.0 (kuromoji). El diccionario de kuromoji es grande: el paquete ocupa unos 41 MB en npm.	**Sí, con descarga aparte.** Un botón «Añadir furigana» que descarga el diccionario la primera vez. Hay que revisar el resultado: algunas palabras tienen más de una lectura.
pinyin-pro	Pone el pinyin, con tonos, a un texto en chino.	MIT. Menos de 1 MB.	**Sí**, si hay gente estudiando chino. Lo mismo que el furigana, pero para hanzi.

Herramienta	Qué aporta	Licencia y tamaño	Veredicto
Tatoeba	Millones de frases de ejemplo traducidas entre idiomas, descargables cada semana. Parte del japonés y del inglés viene del corpus Tanaka, de dominio público.	Las frases, CC BY 2.0 FR (hay que citar la fuente); una parte, CC0. El audio depende de quien lo grabó.	**Sí, como contenido.** Mazos de Explorar con frases reales, o una «frase de ejemplo» al crear una tarjeta de vocabulario. Mejor con las frases CC0, para no tener que citar cada una.
Whisper en el navegador, con Transformers.js	Reconoce la voz sin conexión y sin enviar el audio a nadie. El modelo pequeño pesa unos 39 MB.	Apache-2.0 (Transformers.js).	**Más adelante.** El dictado ya usa la voz del navegador, que es gratis y rápida. Whisper solo valdría en navegadores que no la tienen, y es más lento en móviles modestos.

Datos sin conexión y sincronización

Aquí he encontrado un riesgo real. En modo local, Flaski guarda todo en una sola entrada de `localStorage` (`backend-local.js`) y la vuelve a escribir entera en cada cambio. Los navegadores suelen dejar unos 5 MB en `localStorage`. Con miles de preguntas y su historial de repasos, ese límite puede llegar, y entonces se dejaría de guardar. Con cuenta no pasa: la copia sin conexión ya va en IndexedDB (`snapshot.js`).

Herramienta	Qué aporta	Licencia y tamaño	Veredicto
idb-keyval	Guardar y leer en IndexedDB con dos funciones (`get` y `set`). IndexedDB permite cientos de MB.	Apache-2.0. Muy pequeña, unos 56 kB en npm con todo.	**Sí, ya.** Pasar el modo local a IndexedDB, con una migración que lea lo que haya en `localStorage` la primera vez. Comprobarlo en la prueba de 5.000 preguntas.

Herramienta	Qué aporta	Licencia y tamaño	Veredicto
Dexie	Una base de datos completa sobre IndexedDB, con tablas, índices y consultas.	Apache-2.0. Unos 3 MB en npm.	**No por ahora.** idb-keyval basta para lo que hace hoy Flaski. Dexie valdría si se quisiera consultar sin cargar todo en memoria.
PowerSync	Una base SQLite en el navegador que se sincroniza sola con Supabase: los cambios se guardan en el móvil y se suben cuando hay conexión. Permitiría crear y editar sin conexión, que hoy no se puede.	El cliente es Apache-2.0, pero necesita un servicio de PowerSync entre la app y Supabase.	**No por ahora.** Es cambiar toda la capa de datos y añadir otro servicio. Antes conviene ampliar la cola de cambios que ya existe (`outbox.js`) a crear y editar.

Calidad: accesibilidad, rendimiento y errores

Para que no haya fallos, lo que más rinde es meter en las pruebas automáticas lo que hoy reviso a mano (contraste, zonas táctiles, etiquetas) y enterarte de los errores que pasan en los móviles de la gente. Ninguna de estas herramientas va dentro de la app, salvo la de errores.

Herramienta	Qué aporta	Licencia	Veredicto
@axe-core/playwright	Revisa la accesibilidad de cada pantalla dentro de las pruebas e2e: contraste, botones sin nombre, campos sin etiqueta, orden de los títulos…	MPL-2.0. Solo en `tests/`.	**Sí, ya.** Una prueba que abra cada pantalla en claro y en oscuro y falle si axe encuentra algo grave. Hace automático lo que yo medía a mano.

Herramienta	Qué aporta	Licencia	Veredicto
Lighthouse CI	Mide el rendimiento, la accesibilidad y si la app cumple como PWA, y avisa si algo empeora.	Apache-2.0. Solo en `tests/`.	**Sí**, junto con la prueba de 5.000 preguntas: un límite de tiempo de arranque que no se pueda pasar.
GlitchTip con el SDK de Sentry	Recoge los errores de JavaScript que les salen a otras personas, con el navegador y la pantalla en la que pasaron. GlitchTip es la versión abierta y compatible con Sentry.	GlitchTip, MIT. La versión alojada es gratis hasta 1.000 errores al mes. El SDK de Sentry para navegador es MIT, pero no es pequeño.	**Sí, cuando haya más gente usando la app.** Sin enviar nunca el texto de preguntas ni apuntes, y explicándolo en la política de privacidad. Es la forma de enterarte de un fallo como el del esqueleto del iPhone sin que nadie te lo cuente.
Umami	Estadísticas de uso sin cookies ni datos personales: qué pantallas se usan y cuánto.	MIT. Se puede alojar en tu servidor (Docker y una base de datos) o usar su nube, con un plan gratis.	**Opcional.** Sirve para saber, por ejemplo, si alguien usa el simulacro. Como no usa cookies, no hace falta el aviso de cookies.
web-vitals	Mide cómo de rápida va la app en móviles de verdad: lo que tarda en verse y en responder a un toque.	Apache-2.0.	**Solo con Umami o GlitchTip**, para enviar las medidas a algún sitio.

Para las animaciones que tenemos pendientes no hace falta ninguna librería: las transiciones de CSS y la View Transitions API del navegador bastan. Motion (MIT) solo valdría para algo que no salga con CSS, como arrastrar con inercia.

Plan para Claude Code

Va en tres tandas, de lo que protege lo que ya hay a lo que añade funciones nuevas. Ninguna toca el push. La primera puede ir justo después de los pasos 7A y 7B de las sugerencias.

1. **Paso 8 · Cimientos** (antes de crecer): pasar el modo local a IndexedDB con idb-keyval; pruebas de accesibilidad con axe; prueba de rendimiento con 5.000 preguntas y Lighthouse CI; sustituir fsrs.js por ts-fsrs.
2. **Paso 9 · Crear más rápido:** foto del temario con Tesseract.js; kana con WanaKana; furigana automático con kuroshiro y pinyin con pinyin-pro.
3. **Más adelante:** ajustar FSRS a tu memoria con fsrs-browser; errores en producción con GlitchTip; mazos de frases de Tatoeba; fórmulas con KaTeX.

Paso 8 · Cimientos con herramientas de código abierto

Reglas para cada librería nueva: va en js/vendor/ (nada de npm en la app ni CDN en tiempo de ejecución), con su licencia en licenses/ y una línea en los créditos del README. Si no se usa al arrancar, se carga solo al usarla y no va en SHELL. Si va en SHELL, sube VERSION y ejecuta node version.mjs. Solo commits, nunca push.

1. Modo local en IndexedDB. backend-local.js guarda todo en una sola entrada de localStorage y la reescribe entera en cada cambio; con miles de preguntas puede pasar del límite (unos 5 MB) y dejar de guardar. Pásalo a IndexedDB con idb-keyval (Apache-2.0). La primera vez, migra lo que haya en localStorage y no lo borres hasta comprobar que se ha leído bien. Si al guardar hay un error de cuota, avisa en pantalla en vez de perder datos. Pruebas: migración, recarga y error de cuota simulado.
2. Accesibilidad automática. Añade @axe-core/playwright en tests/ y una prueba que recorra cada pantalla (Hoy, Biblioteca, apunte, editor, tanda, Plan, simulacro, Estadísticas, Ajustes, Explorar, Perfil) en claro y en oscuro, a 375 y 1440 px. Falla con cualquier problema «serious» o «critical». Arregla lo que salga.
3. Rendimiento. Crea un estado de prueba con 5.000 preguntas, 200 apuntes y un año de repasos. Mide el arranque, abrir Biblioteca, el mapa, el buscador y empezar una tanda, con la CPU del móvil frenada 4 veces. Objetivo: todo por debajo de 1 s. Añade Lighthouse CI con un límite que no se pueda pasar. Si el buscador pasa de 100 ms, prueba MiniSearch (MIT).
4. FSRS oficial. Sustituye js/fsrs.js por ts-fsrs (MIT), la librería oficial de open-spaced-repetition, en su versión para el navegador. Antes, guarda los intervalos que da hoy para un juego de casos, y después explica en ESTADO.md cualquier diferencia. No cambies los datos guardados de nadie sin migración.

Al acabar: unitarias y e2e enteras en verde de una vez, y el informe en ESTADO.md con las medidas de antes y después.

```
Paso 9 · Crear más rápido

Las mismas reglas de librerías que en el paso 8.

1. Foto del temario. Al crear un apunte, «Desde una foto»: hace o elige una
foto y la convierte en texto con Tesseract.js (Apache-2.0), en español, sin
servidor. Sus archivos (unos 15 MB) se descargan la primera vez, con un aviso
del tamaño, y quedan guardados para usarlo sin conexión. Usa lo mismo para
los PDF escaneados que hoy no dan texto. El texto sale en el editor para
revisarlo antes de guardar. Prueba e2e con una imagen de prueba.
2. Kana con el teclado normal. En las preguntas de escribir la respuesta en
japonés, WanaKana (MIT) convierte «kanji» en «かんじ» mientras se escribe. Un
interruptor para quitarlo.
3. Furigana y pinyin automáticos. En el editor de tarjetas de japonés,
«Añadir furigana» con kuroshiro y kuromoji (MIT y Apache-2.0): el diccionario
se descarga la primera vez, con aviso del tamaño. En chino, «Añadir pinyin»
con pinyin-pro (MIT). El resultado se puede editar antes de guardar.

Novedades: añade las tres a news.js y regenera capturas. Solo commits, nunca
push.
```

Fuentes

Las versiones, licencias y tamaños de los paquetes son del registro de npm, consultados hoy. Las páginas que he leído están enlazadas en cada tabla; además:

- Lista de herramientas de FSRS (awesome-fsrs)
- Descargas de Tatoeba y sus licencias
- PowerSync con Supabase
- Whisper sin conexión en el navegador